



Programação web

Iniciando com Nest.js

Prof. Diego Cardoso Borda Castro



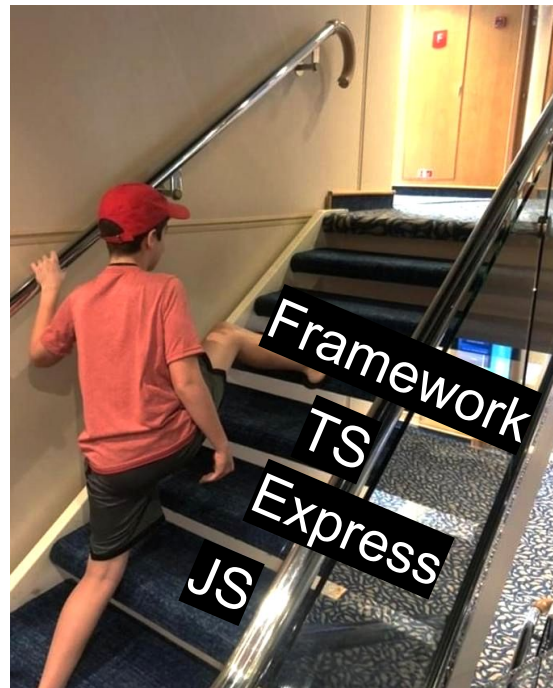
Resumo

- O que é Nest.js
- Controllers
- Providers
- Modules
- Middleware
- Exceptions filters
- Pipes
- Guards
- Interceptors
- Custom decorators
- Fluxo geral
- Hello World



Visão geral

- Desenvolvedor de framework
- Foco do básico
- Foco no conceitos
- Outros frameworks vão surgir





O que é o Nest.js?

- Framework back-end
 - Aplicações escaláveis
 - Criado em 2017
 - Desenvolvimento em camadas
- Incorpora o padrão TypeScript
 - Não é obrigatório
- HTTP Server
 - Express (Padrão)
 - Fastfy (Mais performance)





O que é o Nest.js?

- Ideias de Arquitetura do Angular
 - Injeção dependência
 - Arquitetura modular
 - Artefatos
- Convention Over Configuration (COC)
 - Pastas
 - Nomes arquivos
- Microserviços





O que é o Nest.js?



<https://docs.nestjs.com>



MODULARITY

Streamline upkeep by organizing applications into self-contained modules.



SCALABILITY

Scale seamlessly with an efficient, battle-tested components.



DEPENDENCY INJECTION

Elevate testability with a sophisticated dependency injection system.



TYPE SAFETY

Mitigate errors through the robust type safety features of TypeScript.



RICH ECOSYSTEM

Explore a rich ecosystem offering versatile tools to craft solutions tailored to your needs.



ENTERPRISE-READY

Trusted by thousands of leading companies and organizations worldwide.



MICROSERVICES

Create loosely coupled, independently deployable services for increased agility and scalability.



WEB APPS

Build REST APIs, GraphQL APIs, Queues, and real-time & event-driven applications in no time.

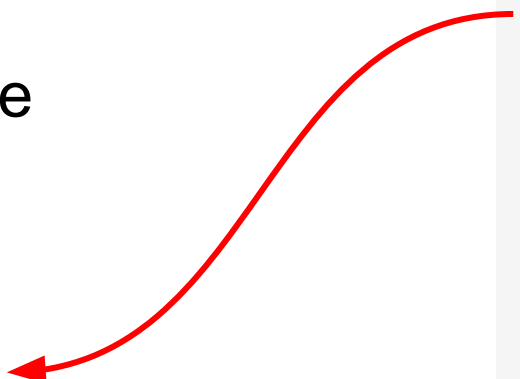


O que é o Nest.js?

- `npm i -g @nestjs/cli`
- `nest new project-name`



OVERVIEW



- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators



O que é o Nest.js?

name	alias	description
application	application	Generate a new application workspace
class	cl	Generate a new class
configuration	config	Generate a CLI configuration file
controller	co	Generate a controller declaration
decorator	d	Generate a custom decorator
filter	f	Generate a filter declaration
gateway	ga	Generate a gateway declaration
guard	gu	Generate a guard declaration
interceptor	itc	Generate an interceptor declaration
interface	itf	Generate an interface
library	lib	Generate a new library within a monorepo
middleware	mi	Generate a middleware declaration
module	mo	Generate a module declaration
pipe	pi	Generate a pipe declaration
provider	pr	Generate a provider declaration
resolver	r	Generate a GraphQL resolver declaration
resource	res	Generate a new CRUD resource
service	s	Generate a service declaration
sub-app	app	Generate a new application within a monorepo

OVERVIEW



First steps

Controllers

Providers

Modules

Middleware

Exception filters

Pipes

Guards

Interceptors

Custom decorators





O que é o Nest.js?

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}

bootstrap();
```

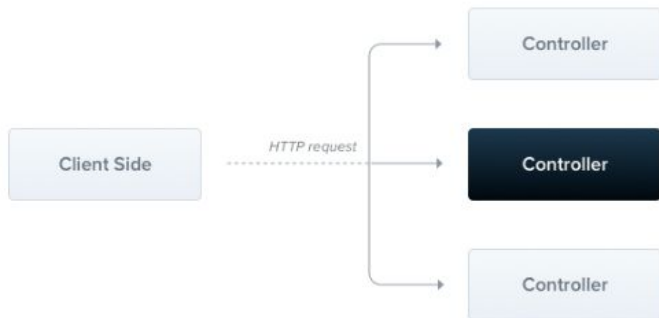
```
const app = await NestFactory.create<NestExpressApplication>(AppModule);
```





Controllers

- Possui as rotas das nossas aplicações
- @ Controller



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Controllers

- Decorators
 - Associação de classes
 - Mapa de roteamento
 - Injeção de dependência
- CRUD generator
 - nest g mo [name]
 - nest g co [name]
 - nest g s [name]
 - nest g resource [name]

OVERVIEW



- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators



Controllers

```
@Controller('users')
export class UsersController {
  constructor(private readonly userService: UsersService) {}

  @Post()
  create(@Body() createUserDto: CreateUserDto) {
    return this.userService.create(createUserDto);
  }

  @Get()
  findAll() {
    return this.userService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.userService.findOne(+id);
  }
}
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





Providers

- Injeção de dependência
- @Injectable
- Gerenciada pelo container Nest IoC
- Injeção baseada em construtor

```
@Controller('cats')
export class CatsController {
  constructor(private catsService: CatsService) {}
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Providers

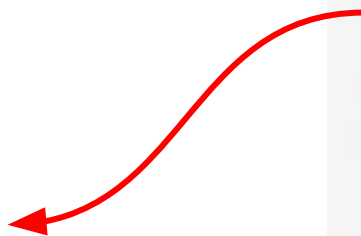
- Injeção baseada em propriedades
- Sem 1001 parâmetros

```
import { Injectable, Inject } from '@nestjs/common';

@Injectable()
export class HttpService<T> {
  @Inject('HTTP_OPTIONS')
  private readonly httpClient: T;
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Providers

- Injeção de dependência

```
import { Injectable } from '@nestjs/common';
import { Cat } from '../interfaces/cat.interface';

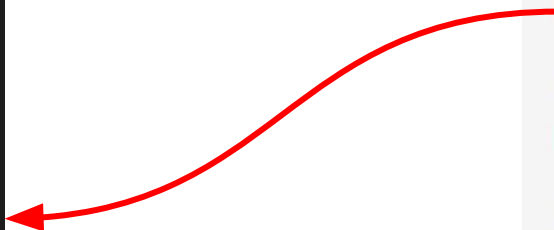
@Injectable()
export class CatsService {
  private readonly cats: Cat[] = [];

  create(cat: Cat) {
    this.cats.push(cat);
  }

  findAll(): Cat[] {
    return this.cats;
  }
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Providers

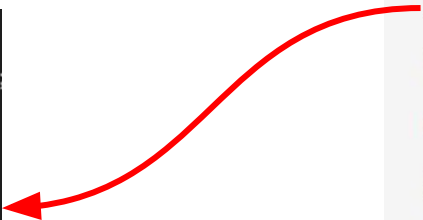
- Registrar os provider

```
import { Module } from '@nestjs/common';
import { CatsController } from '../cats/cats.controller';
import { CatsService } from '../cats/cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class AppModule {}
```

OVERVIEW

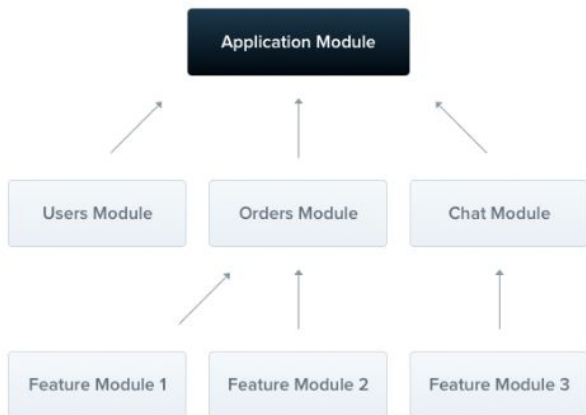
First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





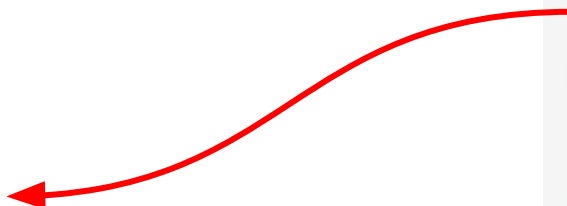
Modules

- Ideia modular do Angular
- Decorator @Module()
- Tudo é um baseado em módulo no Nest



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Modules

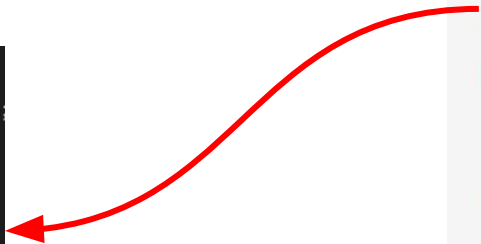
- Decorator @Module()
- Toda aplicação precisa de pelo menos 1 module

```
import { Module } from '@nestjs/common';
import { CatsController } from '../cats/cats.controller';
import { CatsService } from '../cats/cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class AppModule {}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Modules

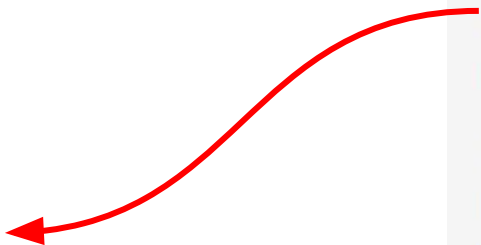
- @Global
 - Injeta em todos os modules

```
import { Module, Global } from '@nestjs/common';
import { CatsController } from './cats.controller';
import { CatsService } from './cats.service';

@Global()
@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





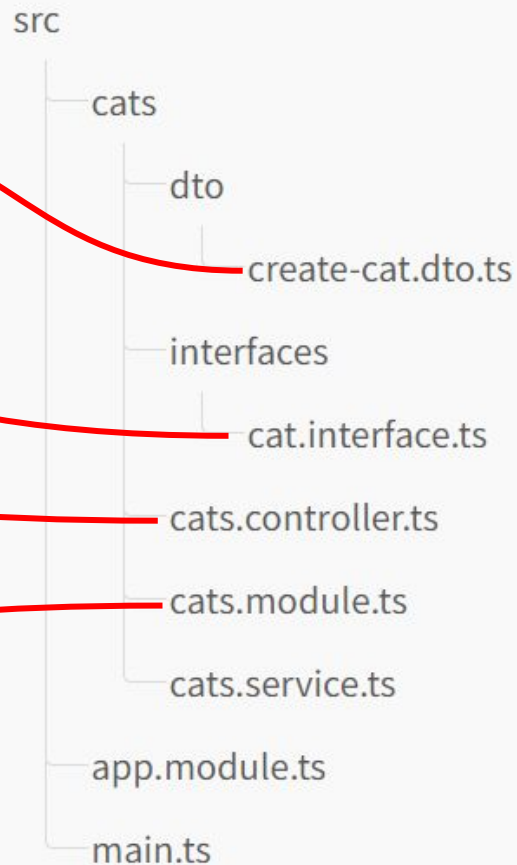
Estrutura do projeto

Objeto de Transferência de Dados

Atributos do objeto

Rotas da nossa API

Organizar e encapsular



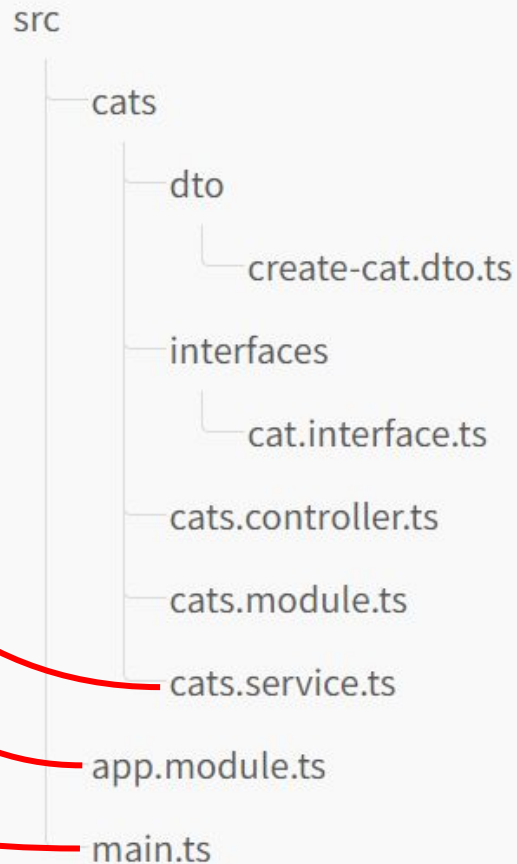


Estrutura do projeto

Regras de negócio

Organizar e encapsular o projeto da api como um todos

Inicia nossa api





Middleware

- Express middleware
- Função chamada antes da execução da rota
- Validações
- NestMiddleware interface



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Middleware

```
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log('Request...');
    next();
  }
}
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware**
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





Exception filters

- O Nest já vem com camada de exceções integrada
- Caso uma exceção não seja tratada
 - Ele captura e manda um mensagem ‘amigável’

Isso não quer dizer que você não deve tratar seus erros

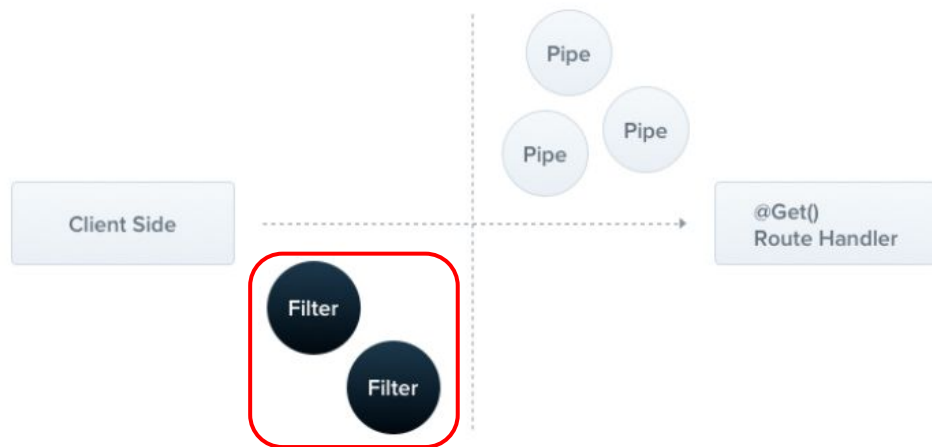
OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





O que é o Nest.js?



OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





O que é o Nest.js?

- Classe de exceção do nest
 - `HttpException`

```
{  
  "statusCode": 403,  
  "message": "Forbidden"  
}
```

```
@Get()  
async findAll() {  
  throw new HttpException('Forbidden', HttpStatus.FORBIDDEN);  
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Exception filters

- Exceção customizada

```
@Get()
async findAll() {
  throw new ForbiddenException();
}
```

```
export class ForbiddenException extends HttpException {
  constructor() {
    super('Forbidden', HttpStatus.FORBIDDEN);
  }
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Exception filters

- @nest/common

- `BadRequestException`
- `UnauthorizedException`
- `NotFoundException`
- `ForbiddenException`
- `NotAcceptableException`
- `RequestTimeoutException`
- `ConflictException`
- `GoneException`
- `HttpVersionNotSupportedException`
- `PayloadTooLargeException`
- `UnsupportedMediaTypeException`
- `UnprocessableEntityException`

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





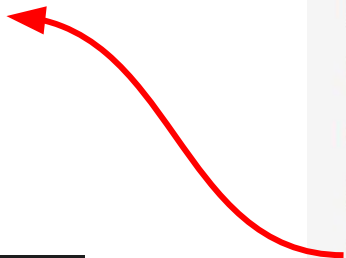
O que é o Nest.js?

- @Injectable() decorator
- PipeTransform interface
 - Transformação de dados
 - Validação de dados
 - Lançar uma exceção
 - DTO

```
@Get(':id')
async findOne(@Param('id', ParseIntPipe) id: number) {
  return this.catsService.findOne(id);
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Pipes

```
@Get('/:uuid')
async findOne(@Param('uuid', new ParseUUIDPipe()) uuid: string) {
  return this.catsService.findOne(uuid);
}
```

```
import { IsString, IsInt } from 'class-validator';

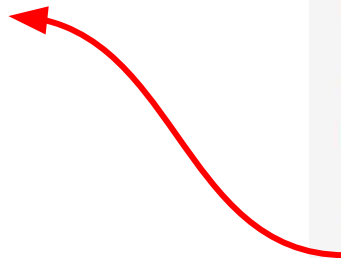
export class CreateCatDto {
  @IsString()
  name: string;

  @IsInt()
  age: number;

  @IsString()
  breed: string;
}
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





Pipes

- ValidationPipe
 - Custom

```
@Post()  
async create(  
  @Body(new ValidationPipe()) createCatDto: CreateCatDto,  
) {  
  this.catsService.create(createCatDto);  
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators

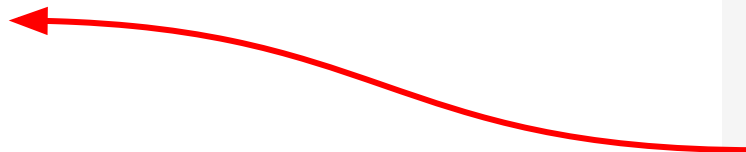




Pipes

- @nest/common

- ValidationPipe
- ParseIntPipe
- ParseFloatPipe
- ParseBoolPipe
- ParseArrayPipe
- ParseUUIDPipe
- ParseEnumPipe
- DefaultValuePipe
- ParseFilePipe



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





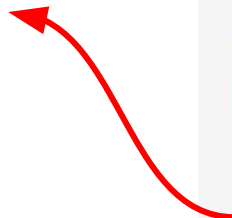
Pipes

- Pipe genérico

```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  app.useGlobalPipes(new ValidationPipe());  
  await app.listen(3000);  
}  
bootstrap();
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





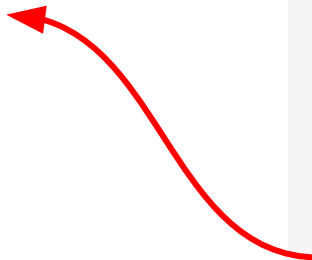
Guard

- @Injectable() decorator
- Única responsabilidade
 - Autorização
 - Google
 - Jwt
 - Etc



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





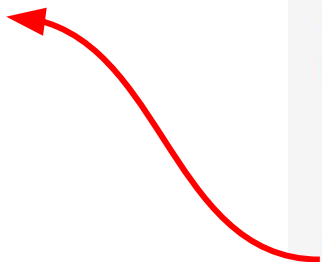
Guard

```
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { Observable } from 'rxjs';

@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(
    context: ExecutionContext,
  ): boolean | Promise<boolean> | Observable<boolean> {
    return true;
  }
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





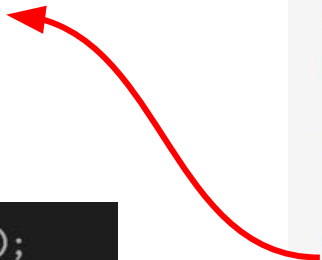
Guard

```
@Controller('cats')
@UseGuards(RolesGuard)
export class CatsController {}
```

```
const app = await NestFactory.create(AppModule);
app.useGlobalGuards(new RolesGuard());
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards**
- Interceptors
- Custom decorators





Interceptors

- Conjunto de recursos
- Orientação baseada a aspectos
 - lógica extra antes/depois do método ...



OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators



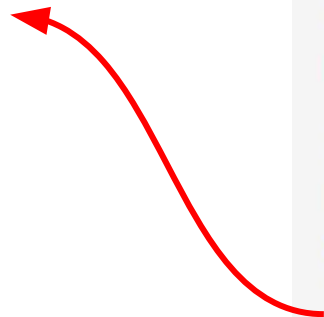


Interceptors

- Conjunto de recursos
- Orientação baseada a aspectos
 - Lógica extra antes/depois do método
 - Transformar o resultado retornado
 - Tratar exceção
 - Substituir função
 - Cache

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Interceptors

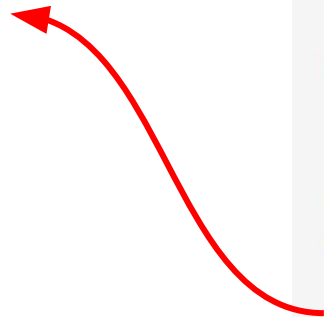
```
@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    console.log('Before...');

    const now = Date.now();
    return next
      .handle()
      .pipe(
        tap(() => console.log(`After... ${Date.now() - now}ms`)),
      );
  }
}
```

```
@UseInterceptors(LoggingInterceptor)
export class CatsController {}
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors**
- Custom decorators





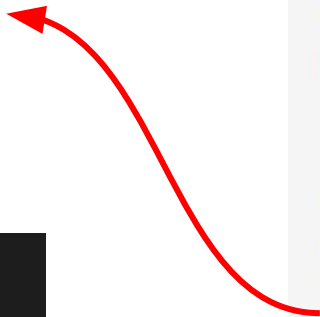
Interceptors

```
@Injectable()
export class ErrorsInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    return next
      .handle()
      .pipe(
        catchError(err => throwError(() => new BadGatewayException())),
      );
  }
}
```

```
const app = await NestFactory.create(AppModule);
app.useGlobalInterceptors(new LoggingInterceptor());
```

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors**
- Custom decorators





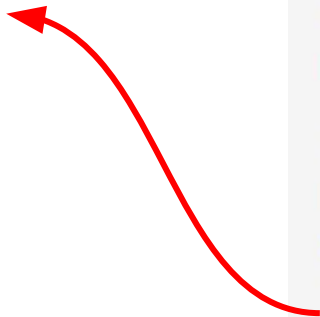
Interceptors

- RxJS
 - isCached
 - Next

```
@Injectable()
export class CacheInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const isCached = true;
    if (isCached) {
      return of([]);
    }
    return next.handle();
  }
}
```

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators





Custom decorators

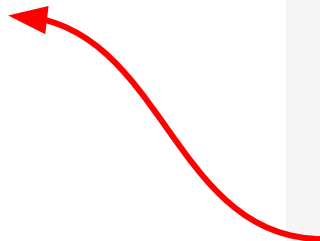
```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export const User = createParamDecorator(
  (data: unknown, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    return request.user;
  },
);
```

```
@Get()
async findOne(@User() user: UserEntity) {
  console.log(user);
}
```

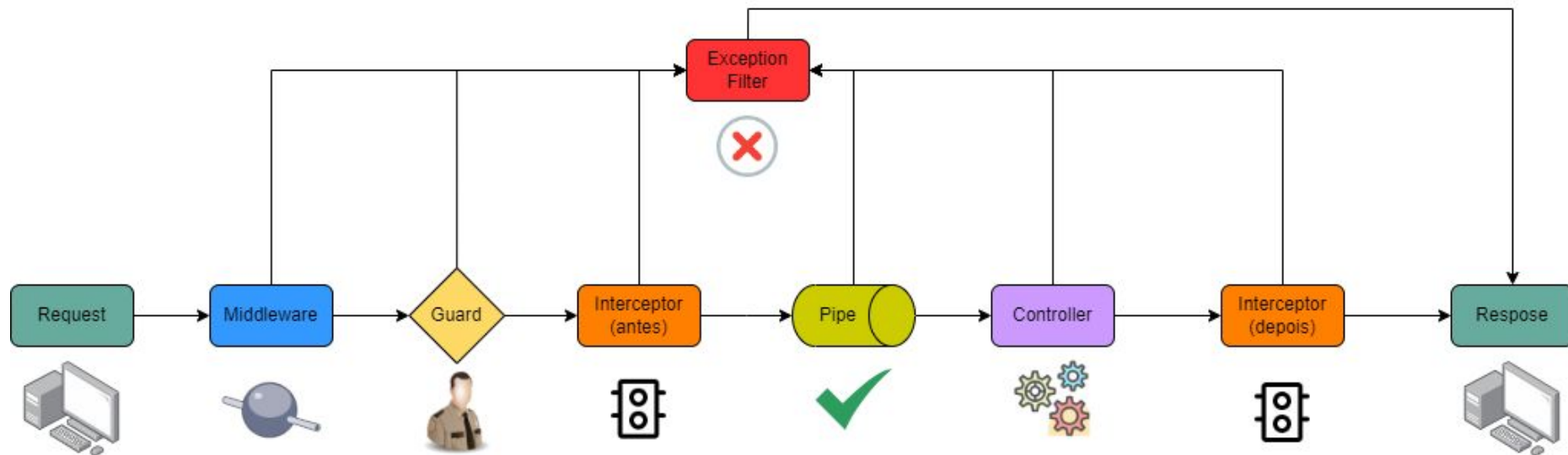
OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes
- Guards
- Interceptors
- Custom decorators





Fluxo geral





Hello World

- `npm i -g @nestjs/cli`
- `nest new hello-world`

```
PS C:\Users\diego\OneDrive\Documentos\cefet\aula 1\Node\semana 5> nest new hello-world
⚡ We will scaffold your app in a few seconds..

? Which package manager would you ❤️ to use? yarn
CREATE hello-world/.eslintrc.js (688 bytes)
CREATE hello-world/.prettierrc (54 bytes)
CREATE hello-world/nest-cli.json (179 bytes)
CREATE hello-world/package.json (2019 bytes)
CREATE hello-world/README.md (3420 bytes)
CREATE hello-world/tsconfig.build.json (101 bytes)
CREATE hello-world/tsconfig.json (567 bytes)
CREATE hello-world/src/app.controller.ts (286 bytes)
CREATE hello-world/src/app.module.ts (259 bytes)
CREATE hello-world/src/app.service.ts (150 bytes)
CREATE hello-world/src/main.ts (216 bytes)
CREATE hello-world/src/app.controller.spec.ts (639 bytes)
CREATE hello-world/test/jest-e2e.json (192 bytes)
CREATE hello-world/test/app.e2e-spec.ts (654 bytes)
```

SEMANA 5

hello-world

> node_modules

src

TS app.controller.spec.ts

TS app.controller.ts

TS app.module.ts

TS app.service.ts

TS main.ts

> test

🔗 .eslintrc.js

{ } .prettierrc

{ } nest-cli.json

{ } package.json

📄 README.md

{ } tsconfig.build.json

TS tsconfig.json



Hello World

```
import { Controller, Get } from '@nestjs/common';
import { AppService } from './app.service';

@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}
bootstrap();
```

SEMANA 5

hello-world

> node_modules

src

TS app.controller.spec.ts

TS app.controller.ts

TS app.module.ts

TS app.service.ts

TS main.ts

> test

🔗 .eslintrc.js

{ } .prettierrc

{ } nest-cli.json

{ } package.json

📖 README.md

{ } tsconfig.build.json

TS tsconfig.json



Hello World

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';

@Module({
  imports: [],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```

SEMANA 5

hello-world

> node_modules

src

TS app.controller.spec.ts

TS app.controller.ts

TS app.module.ts

TS app.service.ts

TS main.ts

> test

🔗 .eslintrc.js

{ } .prettierrc

{ } nest-cli.json

{ } package.json

📖 README.md

{ } tsconfig.build.json

TS tsconfig.json



Gerador de CRUD

```
● PS C:\Users\diego\OneDrive\Documentos\cefet\aula 1\Node\semana 5\hello-world> nest g res aluno
⊗ ? What transport layer do you use? REST API
? Would you like to generate CRUD entry points? Yes
CREATE src/aluno/aluno.controller.ts (938 bytes)
CREATE src/aluno/aluno.controller.spec.ts (586 bytes)
CREATE src/aluno/aluno.module.ts (257 bytes)
CREATE src/aluno/aluno.service.ts (647 bytes)
CREATE src/aluno/aluno.service.spec.ts (471 bytes)
CREATE src/aluno/dto/create-aluno.dto.ts (32 bytes)
CREATE src/aluno/dto/update-aluno.dto.ts (177 bytes)
CREATE src/aluno/entities/aluno.entity.ts (23 bytes)
UPDATE package.json (2053 bytes)
UPDATE src/app.module.ts (322 bytes)
✓ Packages installed successfully.
○ PS C:\Users\diego\OneDrive\Documentos\cefet\aula 1\Node\semana 5\hello-world> █
```



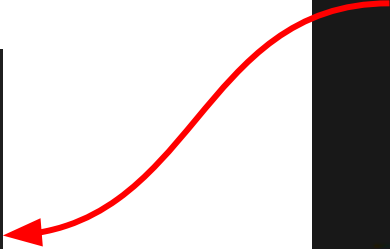
Gerador de CRUD

- DTO
 - Entrada de dados
- Entidade
 - Reflexo da minha tabela

```
@Controller('aluno')
export class AlunoController {
  constructor(private readonly alunoService: AlunoService) {}

  @Post()
  create(@Body() createAlunoDto: CreateAlunoDto) {
    return this.alunoService.create(createAlunoDto);
  }

  @Get()
  findAll() {
    return this.alunoService.findAll();
  }
}
```



src	
aluno	
dto	
create-aluno.dto.ts	U
update-aluno.dto.ts	U
entities	
aluno.entity.ts	U
aluno.controller.spec.ts	U
aluno.controller.ts	U
aluno.module.ts	U
aluno.service.spec.ts	U
aluno.service.ts	U
app.controller.spec.ts	U
app.controller.ts	U
app.module.ts	U
app.service.ts	U
main.ts	U



Gerador de CRUD

- Já importa o módulo de aluno

```
import { Module } from '@nestjs/common';
import { AppController } from '../app.controller';
import { AppService } from '../app.service';
import { AlunoModule } from '../aluno/aluno.module';

@Module({
  imports: [AlunoModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

